# BEE 4750 Homework 1: Introduction to Using Julia

**Name**: Sarah Vafiadis, Ella Hartmanis, Nathan Rhoads

**ID**: sv439, ,

**Due Date**

Thursday, 9/11/25, 9:00pm

## Overview

### Instructions

- Problem 1 consist of a series of code snippets for you to interpret and debug. You will be asked to identify relevant error(s) and fix the code.
- Problem 2 asks you to write code to implement several simple functions using more general programming and some which are unique to Julia (which might mean that you need to look at the documentation or use Julia's help function, which is accessed with `?`, e.g. `?sum` for help with the `sum()` function) or to analyze Julia syntax.
- Problem 3 asks you to convert a verbal description of a wastewater treatment system into a Julia function, and then to use that function to explore the impact of different wastewater allocation strategies.

### Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```
  Activating project at `c:\Users\sarah\OneDrive\Documents\GitHub\bee4750-hw1-sarah-and-natha
   Installed HypergeometricFunctions  v0.3.28
   Installed EarCut_jll         v2.2.4+0
   Installed PDMats            v0.11.35
   Installed StatsFuns          v1.5.0
   Installed NetworkLayout        v0.4.10
   Installed ArnoldiMethod        v0.4.0
   Installed OffsetArrays        v1.17.0
   Installed Unitful            v1.23.1
   Installed StaticArrays         v1.9.14
   Installed Rmath_jll          v0.5.1+0
   Installed StaticArraysCore       v1.4.3
   Installed Ratios            v0.4.5
   Installed SpecialFunctions       v2.5.1
   Installed GeometryTypes        v0.8.5
   Installed ColorTypes          v0.11.5
   Installed GraphRecipes         v0.5.14
   Installed Extents            v0.1.6
   Installed AbstractTrees         v0.4.5
   Installed Inflate            v0.1.5
   Installed OpenSpecFun_jll       v0.5.6+0
   Installed ColorVectorSpace       v0.10.0
   Installed ChainRulesCore        v1.25.2
   Installed Graphs            v1.13.0
   Installed SimpleTraits         v0.9.4
   Installed Interpolations        v0.16.1
   Installed Adapt             v4.3.0
   Installed Rmath             v0.8.0
   Installed IterTools           v1.10.0
   Installed GeometryBasics        v0.5.10
   Installed FillArrays          v1.13.0
   Installed WoodburyMatrices       v1.0.0
   Installed AxisAlgorithms        v1.1.0
   Installed QuadGK            v2.11.2
   Installed Distributions         v0.25.120
Precompiling project...
   1591.6 ms   Extents
   1590.8 ms   Ratios
   1526.6 ms   StaticArraysCore
   1585.5 ms   Inflate
   1688.2 ms   IterTools
   1674.4 ms   AbstractTrees
   1582.8 ms   Adapt
```

```
 2133.2 ms    OffsetArrays
 1724.8 ms    FillArrays
 1471.6 ms    Rmath_jll
 1808.6 ms    EarCut_jll
 1917.8 ms    SimpleTraits
 2179.8 ms    OpenSpecFun_jll
 2300.4 ms    ChainRulesCore
 1151.8 ms    Ratios → RatiosFixedPointNumbersExt
 3629.8 ms    SuiteSparse
 3709.8 ms    WoodburyMatrices
 1362.4 ms    Adapt → AdaptSparseArraysExt
 2384.8 ms    QuadGK
 4401.4 ms    ColorTypes
 1485.1 ms    OffsetArrays → OffsetArraysAdaptExt
 1172.6 ms    FillArrays → FillArraysStatisticsExt
 1254.8 ms    FillArrays → FillArraysSparseArraysExt
 2077.0 ms    Rmath
 1173.5 ms    AxisAlgorithms
 1487.4 ms    PDMats
 2000.3 ms    ChainRulesCore → ChainRulesCoreSparseArraysExt
 2373.3 ms    LogExpFunctions → LogExpFunctionsChainRulesCoreExt
 1274.3 ms    FillArrays → FillArraysPDMatsExt
          Conda
 4608.5 ms    SpecialFunctions
 3166.3 ms    ColorVectorSpace
 2020.8 ms    HypergeometricFunctions
 9503.6 ms    StaticArrays
 2962.3 ms    ColorVectorSpace → SpecialFunctionsExt
 5950.7 ms    Colors
 3617.7 ms    SpecialFunctions → SpecialFunctionsChainRulesCoreExt
          IJulia
 1463.9 ms    Adapt → AdaptStaticArraysExt
 3323.3 ms    StatsFuns
 2501.2 ms    StaticArrays → StaticArraysStatisticsExt
 2669.6 ms    StaticArrays → StaticArraysChainRulesCoreExt
 3478.7 ms    ArnoldiMethod
 2882.1 ms    StatsFuns → StatsFunsChainRulesCoreExt
 3165.0 ms    GeometryTypes
 3470.4 ms    Interpolations
 6980.9 ms    ColorSchemes
 6550.6 ms    Distributions
 7231.9 ms    Graphs
23318.6 ms    Unitful
```

```
   3635.8 ms    Distributions → DistributionsChainRulesCoreExt
   4471.0 ms    Distributions → DistributionsTestExt
   3213.6 ms    Unitful → PrintfExt
  14670.1 ms    GeometryBasics
   4338.7 ms    Interpolations → InterpolationsUnitfulExt
   2767.3 ms    UnitfulLatexify
   3932.9 ms    NetworkLayout
  11760.8 ms    PlotUtils
   2546.6 ms    NetworkLayout → NetworkLayoutGraphsExt
   4909.0 ms    RecipesPipeline
   5089.4 ms    PlotThemes
   4292.2 ms    GraphRecipes
  62394.7 ms    Plots
   5973.3 ms    Plots → UnitfulExt
   5945.0 ms    Plots → GeometryBasicsExt
           Plots → IJuliaExt
  63 dependencies successfully precompiled in 111 seconds. 174 already precompiled.

The following 2 direct dependencies failed to precompile:

IJuliaExt

Failed to precompile IJuliaExt [2f4121a4-3b3a-5ce6-9c5e-1f2673ce168a] to "C:\\Users\\sarah\\
ERROR: LoadError: IJulia not properly installed. Please run Pkg.build("IJulia")
Stacktrace:
 [1] error(s::String)
   @ Base .\error.jl:35
 [2] top-level scope
   @ C:\Users\sarah\.julia\packages\IJulia\eenvU\src\IJulia.jl:50
 [3] include
   @ .\Base.jl:557 [inlined]
 [4] include_package_for_output(pkg::Base.PkgId, input::String, depot_path::Vector{String}, c
   @ Base .\loading.jl:2881
 [5] top-level scope
   @ stdin:6
in expression starting at C:\Users\sarah\.julia\packages\IJulia\eenvU\src\IJulia.jl:1
in expression starting at stdin:6
ERROR: LoadError: Failed to precompile IJulia [7073ff75-c697-5162-941a-fcdaad2a7d2a] to "C:\\
Stacktrace:
  [1] error(s::String)
    @ Base .\error.jl:35
  [2] compilecache(pkg::Base.PkgId, path::String, internal_stderr::IO, internal_stdout::IO, I
    @ Base .\loading.jl:3174
```

```
  [3] (::Base.var"#1110#1111"{Base.PkgId})()
    @ Base .\loading.jl:2579
  [4] mkpidlock(f::Base.var"#1110#1111"{Base.PkgId}, at::String, pid::Int32; kwopts::@Kwargs
    @ FileWatching.Pidfile C:\Users\sarah\.julia\juliaup\julia-1.11.5+0.x64.w64.mingw32\share
  [5] #mkpidlock#6
    @ C:\Users\sarah\.julia\juliaup\julia-1.11.5+0.x64.w64.mingw32\share\julia\stdlib\v1.11\
  [6] trymkpidlock(::Function, ::Vararg{Any}; kwargs::@Kwargs{stale_age::Int64})
    @ FileWatching.Pidfile C:\Users\sarah\.julia\juliaup\julia-1.11.5+0.x64.w64.mingw32\share
  [7] #invokelatest#2
    @ .\essentials.jl:1057 [inlined]
  [8] invokelatest
    @ .\essentials.jl:1052 [inlined]
  [9] maybe_cachefile_lock(f::Base.var"#1110#1111"{Base.PkgId}, pkg::Base.PkgId, srcpath::Str
    @ Base .\loading.jl:3698
 [10] maybe_cachefile_lock
    @ .\loading.jl:3695 [inlined]
 [11] _require(pkg::Base.PkgId, env::Nothing)
    @ Base .\loading.jl:2565
 [12] __require_prelocked(uuidkey::Base.PkgId, env::Nothing)
    @ Base .\loading.jl:2388
 [13] #invoke_in_world#3
    @ .\essentials.jl:1089 [inlined]
 [14] invoke_in_world
    @ .\essentials.jl:1086 [inlined]
 [15] _require_prelocked
    @ .\loading.jl:2375 [inlined]
 [16] _require_prelocked
    @ .\loading.jl:2374 [inlined]
 [17] macro expansion
    @ .\lock.jl:273 [inlined]
 [18] require(uuidkey::Base.PkgId)
    @ Base .\loading.jl:2371
 [19] top-level scope
    @ C:\Users\sarah\.julia\packages\Plots\gYkEG\ext\IJuliaExt.jl:6
 [20] include
    @ .\Base.jl:557 [inlined]
 [21] include_package_for_output(pkg::Base.PkgId, input::String, depot_path::Vector{String},
    @ Base .\loading.jl:2881
 [22] top-level scope
    @ stdin:6
in expression starting at C:\Users\sarah\.julia\packages\Plots\gYkEG\ext\IJuliaExt.jl:1
in expression starting at stdin:6
IJulia
```

```
Failed to precompile IJulia [7073ff75-c697-5162-941a-fcdaad2a7d2a] to "C:\\Users\\sarah\\.jul
ERROR: LoadError: IJulia not properly installed. Please run Pkg.build("IJulia")
Stacktrace:
 [1] error(s::String)
   @ Base .\error.jl:35
 [2] top-level scope
   @ C:\Users\sarah\.julia\packages\IJulia\eenvU\src\IJulia.jl:50
 [3] include
   @ .\Base.jl:557 [inlined]
 [4] include_package_for_output(pkg::Base.PkgId, input::String, depot_path::Vector{String}, c
   @ Base .\loading.jl:2881
 [5] top-level scope
   @ stdin:6
in expression starting at C:\Users\sarah\.julia\packages\IJulia\eenvU\src\IJulia.jl:1
in expression starting at stdin:6
```

Standard Julia practice is to load all needed packages at the top of a file. If you need to load any additional packages in any assignments beyond those which are loaded by default, feel free to add a `using` statement, though you may need to install the package.

```
using Random
using Plots
using GraphRecipes
using LaTeXStrings
using Distributions
```

```
# this sets a random seed, which ensures reproducibility of random number generation. You sho
Random.seed!(1)
```

```
TaskLocalRNG()
```

## Problems (Total: 30 Points)

### Problem 1 (12 points)

The following subproblems all involve code snippets that require debugging. You are encouraged to use online resources (*e.g.* Julia documentation and forums, Stack Overflow, Reddit, etc) to help find diagnose error messages and find solutions, but make sure that you clearly document which resources you used and how you used them.

### Problem 1.1

You've been tasked with writing code to identify the minimum value in an array. You cannot use a predefined function. Your colleague suggested the function below, but it does not return the minimum value.

```
function minimum(array)
    # initialize the minimum value counter
    min_value = 0
    # update minimum values
    for i in 1:length(array)
        if array[i] < min_value
            min_value = array[i]
        end
    end
    # return found minimum
    return min_value
end

array_values = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
@show minimum(array_values);
```

```
minimum(array_values) = 0
```

What is the logical flaw in the code? Fix it and show that your fixed code solves the problem.

### Problem 1.2

Your team is trying to compute the average grade for your class, but the following code produces an `UndefVarError`.

```
# enter student grade vector
student_grades = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
# compute class average
function class_average(grades)
  average_grade = mean(student_grades)
  return average_grade
end

@show average_grade;
```

```
UndefVarError: UndefVarError: `average_grade` not defined in `Main`
Suggestion: check for spelling errors or missing imports.
UndefVarError: `average_grade` not defined in `Main`

Suggestion: check for spelling errors or missing imports.



Stacktrace:

 [1] macro expansion

   @ show.jl:1232 [inlined]

 [2] top-level scope

   @ c:\Users\sarah\OneDrive\Documents\GitHub\bee4750-hw1-sarah-and-nathan\jl_notebook_cell_
```

What does this `UndefVarError` mean? Why does this code produce one? Fix the error and solve the problem.

### Problem 1.3

Your team wants to know the expected payout of an old Italian dice game called *passadieci* (which was analyzed by Galileo as one of the first examples of a rigorous study of probability). The goal of passadieci is to get at least an 11 from rolling three fair, six-sided dice. Your strategy is to compute the average wins from 1,000 trials, but the code you've written below produces a `MethodError`.

```julia
# function to simulate a passadieci roll: 3 6-sided dice
function passadieci()
    # this rand() call samples 3 values from the vector [1, 6]
    roll = rand(1:6, 3)
    return roll
end
# set number of trials and initialize outcome vector
n_trials = 1_000
outcomes = zero(n_trials)
# simulate number of passadieci rolls and count wins
for i = 1:n_trials
    outcomes[i] = (sum(passadieci()) > 11)
end
```

```
win_prob = sum(outcomes) / n_trials # compute average number of wins
@show win_prob;
```

```
MethodError: MethodError: no method matching setindex!(::Int64, ::Bool, ::Int64)
The function `setindex!` exists, but no method is defined for this combination of argument ty
MethodError: no method matching setindex!(::Int64, ::Bool, ::Int64)

The function `setindex!` exists, but no method is defined for this combination of argument ty



Stacktrace:

 [1] top-level scope

   @ c:\Users\sarah\OneDrive\Documents\GitHub\bee4750-hw1-sarah-and-nathan\jl_notebook_cell_c
```

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

### Problem 1.4

You're interested in writing some code to remove the mean of a vector from all of its components. You've written the following code and tried to test it on a random vector, but your code returns a `MethodError`.

```
# function to remove mean from a vector
function remove_mean(vect)
    # fucntion to compute the mean
    function compute_mean(vect)
        element_sum = 0 # initialize sum
        # compute mean and return
        for v in vect
            element_sum += v
        end
        return element_sum / length(vect)
    end

    m = compute_mean(vect) # compute mean
    # return demeaned vector
    return vect - m
```

```
end

random_vect = rand(1_000)
@show remove_mean(random_vect)
```

```
MethodError: MethodError: no method matching -(::Vector{Float64}, ::Float64)
For element-wise subtraction, use broadcasting with dot syntax: array .- scalar
The function `-` exists, but no method is defined for this combination of argument types.

Closest candidates are:
  -(!Matched::Missing, ::Number)
   @ Base missing.jl:123
  -(!Matched::ChainRulesCore.NoTangent, ::Any)
   @ ChainRulesCore C:\Users\sarah\.julia\packages\ChainRulesCore\XAgYn\src\tangent_arithmeti
  -(::Any, !Matched::ChainRulesCore.NotImplemented)
   @ ChainRulesCore C:\Users\sarah\.julia\packages\ChainRulesCore\XAgYn\src\tangent_arithmeti
  ...

MethodError: no method matching -(::Vector{Float64}, ::Float64)

For element-wise subtraction, use broadcasting with dot syntax: array .- scalar

The function `-` exists, but no method is defined for this combination of argument types.



Closest candidates are:

  -(!Matched::Missing, ::Number)

   @ Base missing.jl:123

  -(!Matched::ChainRulesCore.NoTangent, ::Any)

   @ ChainRulesCore C:\Users\sarah\.julia\packages\ChainRulesCore\XAgYn\src\tangent_arithmeti

  -(::Any, !Matched::ChainRulesCore.NotImplemented)

   @ ChainRulesCore C:\Users\sarah\.julia\packages\ChainRulesCore\XAgYn\src\tangent_arithmeti

  ...
```

```
Stacktrace:

 [1] remove_mean(vect::Vector{Float64})

   @ Main c:\Users\sarah\OneDrive\Documents\GitHub\bee4750-hw1-sarah-and-nathan\jl_notebook_

 [2] macro expansion

   @ show.jl:1232 [inlined]

 [3] top-level scope

   @ c:\Users\sarah\OneDrive\Documents\GitHub\bee4750-hw1-sarah-and-nathan\jl_notebook_cell_
```

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

### Problem 2 (18 points)

Cheap Plastic Products, Inc. is operating a plant that produces $100\text{m}^3$/day of wastewater that is discharged into Pristine Brook. The wastewater contains $1\text{kg/m}^3$ of YUK, a toxic substance. The US Environmental Protection Agency has imposed an effluent standard on the plant prohibiting discharge of more than 20kg/day of YUK into Pristine Brook.

Cheap Plastic Products has analyzed two methods for reducing its discharges of YUK. Method 1 is land disposal, which costs $X_1^2/20$ dollars per day, where $X_1$ is the amount of wastewater disposed of on the land ($\text{m}^3$/day). With this method, 20% of the YUK applied to the land will eventually drain into the stream (*i.e.*, 80% of the YUK is removed by the soil).

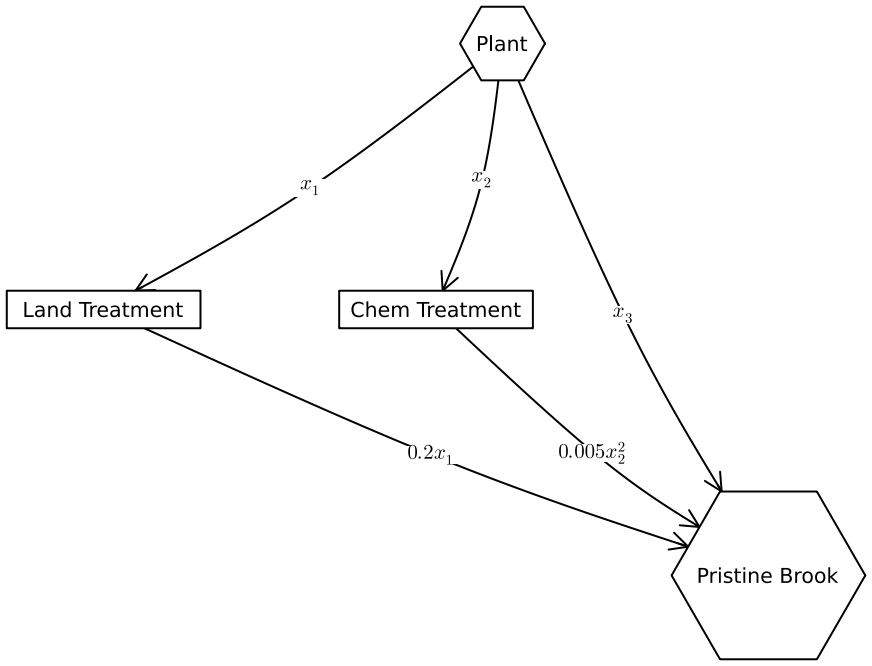
Method 2 is a chemical treatment procedure which costs \$1.50 per $\text{m}^3$ of wastewater treated. The chemical treatment has an efficiency of $e = 1 - 0.005X_2$, where $X_2$ is the quantity of wastewater ($\text{m}^3$/day) treated. For example, if $X_2 = 50\text{m}^3$/day, then $e = 1 - 0.005(50) = 0.75$, so that 75% of the YUK is removed.

Cheap Plastic Products is wondering how to allocate their wastewater between these three disposal and treatment methods (land disposal, chemical treatment, and direct disposal) to meet the effluent standard while keeping costs manageable.

The flow of wastewater through this treatment system is shown in Figure 1. Modify the edge labels (by editing the `edge_labels` dictionary in the code producing Figure 1) to show how

the wastewater allocations result in the final YUK discharge into Pristine Brook. For the `edge_label` dictionary, the tuple $(i, j)$ corresponds to the arrow going from node $i$ to node $j$. The syntax for any entry is `(i, j) => "label text"`, and the label text can include mathematical notation if the string is prefaced with an `L`, as in `L"x_1"` will produce $x_1$.

```
A = [0 1 1 1;
    0 0 0 1;
    0 0 0 1;
    0 0 0 0]

names = ["Plant", "Land Treatment", "Chem Treatment", "Pristine Brook"]
# modify this dictionary to add labels
edge_labels = Dict((1, 2) => L"x_1", (1,3) => L"x_2", (1, 4) => L"x_3",(2, 4) => L"0.2x_1",(
shapes=[:hexagon, :rect, :rect, :hexagon]
xpos = [0, -1.5, -0.25, 1]
ypos = [1, 0, 0, -1]

p = graphplot(A, names=names,edgelabel=edge_labels, markersize=0.15, markershapes=shapes, mar
display(p)
```



### Problem 2.1

Formulate a mathematical model for the treatment cost and the amount of YUK that will be discharged into Pristine Brook based on the wastewater allocations. This is best done with some equations and supporting text explaining the derivation. Make sure you include, as additional equations in the model, any needed constraints on relevant values. You can find some basics on writing mathematical equations using the LaTeX typesetting syntax here, and a cheatsheet with LaTeX commands can be found on the course website's Resources page.

### Our solution

#### Variables and constraints

Volume of wastewater disposed on land: $x_1 \geq 0$

Volume of wastewater chemically treated: $x_2 \geq 0$

Volume of wastewater disposed directly from plant: $x_3 \geq 0$

#### Equations

Overall wastewater flow: $x_1 + x_2 + x_3 = 100$

The total volume of wastewater analyzed in this problem is 100 $m^3$, so all three wastewater flows should be equal to that value.

YUK discharged into Pristine Brook: $0.2x_1 + 0.005x_2^2 + x_3 \geq 20$

When land treatment is used, 20% of the volume treated is drained into Pristine Brook. Therefore, the volume treated ($x_1$), should be multiplied by 0.2. Chemical treatment has an efficiency of $e = 1 - 0.005x_2$. This gives the percentage of wastewater divered from Pristine Brook. To calculate the amount entering the brook, the equation $1-e$ should be used. When the equation for e is substituted into this equation, the result is $0.005x_2^2$. $x_3$ is the amount of wastewater released directly from the plant into the brook. All of these values are multiplied by 1 kg/$m^3$, the amount of YUK in each $m^3$ of water. Since this value is 1, the overall equation doesn't change. Finally, no more than 20 kg of YUK may be directed into the brook each day, so this equation must be less than or equal to 20.

Cost:

$ x_1^2/2 + 1.5x_2 = C$

This equation accounts for the costs of land and chemical treatment as given in the problem statement.

### Problem 2.2

Implement your systems model as a Julia function which computes the resulting YUK discharge and cost for a particular treatment plan. You can return multiple values from a function with a tuple, as in:

```
function multiple_return_values(x, y)
    return (x+y, x*y)
end

a, b = multiple_return_values(2, 5)
@show a;
@show b;
```

```
a = 7
b = 10
```

To evalute the function over vectors of inputs, you can *broadcast* the function by adding a decimal . before the function arguments and accessing the resulting values by writing a *comprehension* to loop over the individual outputs in the vector:

```
x = [1, 2, 3, 4, 5]
y = [6, 7, 8, 9, 10]

output = multiple_return_values.(x, y)
a = [out[1] for out in output]
b = [out[2] for out in output]
@show a;
@show b;
```

```
a = [7, 9, 11, 13, 15]
b = [6, 14, 24, 36, 50]
```

### Our solution

```
function calculate_flows(x_1,x_2)

    # x_3 is dependent on x_1 and x_2 inputs in order to minimize cost
    x_3 = 100 - x_1 - x_2

    # These equations are pulled directly from the system designed in 2.1
```

```
    yuk_dis = 0.2*x_1 + 0.005*x_2^2 + x_3 - 20
    cost = (x_1^2)/2 + 1.5*x_2

    # Check that YUK discharge into Pristine Brook is less than 20 kg/day; @assert command fo
    @assert yuk_dis <= 20

    # Check that all flows are non-negative
    @assert x_1 >= 0
    @assert x_2 >= 0
    @assert x_3 >= 0

    return (yuk_dis, cost, x_1, x_2, x_3)
end

# Unpacks variables to be displayed; values input in calculate_flows may be changed to find o
(yuk_dis, cost, x_1, x_2, x_3) = calculate_flows(40,40)

# Displays solution variables
@show yuk_dis
@show cost
@show x_1
@show x_2
@show x_3
```

```
yuk_dis = 16.0
cost = 860.0
x_1 = 40
x_2 = 40
x_3 = 20

20
```

Make sure you comment your code appropriately to make it clear what is going on and why.

### Problem 2.3

Use your function to experiment with 1,000 different combinations of wastewater discharge and treatment. You can do this with either a grid search or by sampling from a Dirichlet distribution (a Dirichlet$(1, n)$ distribution will generate uniformly-weighted $n$-dimensional vectors whose components add up to 1; see the `Distributions.jl` documentation for how to

sample from probability distributions in Julia). Plot the results of these experiments. Do any satisfy the YUK effluent standard (plot this as well as a dashed red line). What was the cost of solutions satisfying the standard? What can you say about the tradeoff between treatment cost and YUK discharge? You don't have to find an "optimal" solution to this problem, but what do you think would be needed to find a better solution?

### Problem 2.4

Find the strategies which minimize cost and YUK discharge (these will be different strategies) analytically and find the values of the objective metrics. Plot these values in the plot that you created for Problem 2.3. How do their values compare to the spread of values that you found in that problem? Would you select either of them (explain why or why not)?

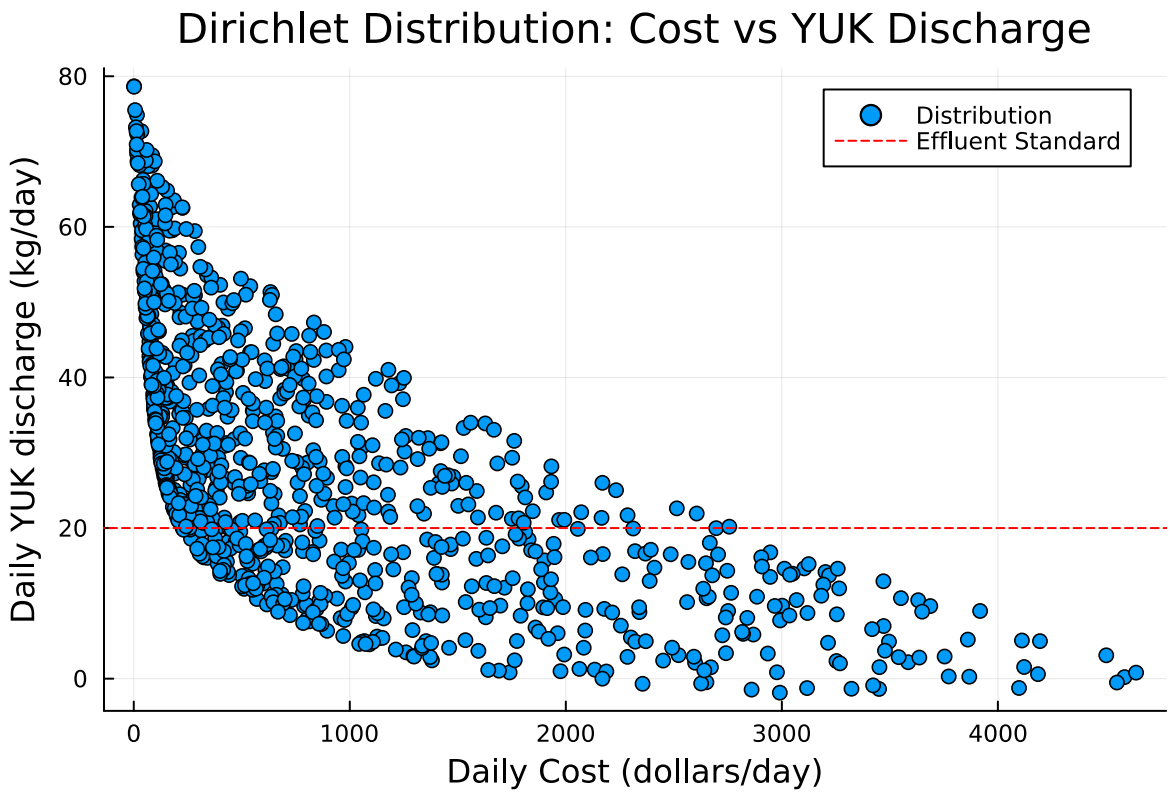
## References

List any external references consulted, including classmates.

### Our Solution

```
using Plots, Distributions

function calculate_flows(x_1,x_2)

    # x_3 is dependent on x_1 and x_2 inputs in order to minimize cost
    x_3 = 100 - x_1 - x_2

    # These equations are pulled directly from the system designed in 2.1
    yuk_dis = 0.2*x_1 + 0.005*x_2^2 + x_3 - 20
    cost = (x_1^2)/2 + 1.5*x_2

    # Check that YUK discharge into Pristine Brook is less than 20 kg/day; @assert command fo
    # Commented out asseration meeting effluent standard to include solutions that may not me

    # @assert yuk_dis <= 20

    # Check that all flows are non-negative
    @assert x_1 >= 0
    @assert x_2 >= 0
    @assert x_3 >= 0

    return (yuk_dis, cost, x_1, x_2, x_3)
```

```
end

# 1000 random samples of x_1, x_2, x_3 generated; although x_3 is calculated in function
# I found it easier to also include it here when summing to 100
d = Dirichlet(ones(3))
data = rand(d, 1000)

# Scale samples so each sums to 100
data_100 = 100 .* data

x = [1, 2, 3, 4, 5]
y = [6, 7, 8, 9, 10]

# Run function on all 1000 combinations of x_1,x_2,x_3
output = calculate_flows.(data_100[1,1:1000], data_100[2,1:1000])

# Extract results for plotting
yuk_dis = [out[1] for out in output]
cost = [out[2] for out in output]
x_1 = [out[3] for out in output]
x_2 = [out[4] for out in output]
x_2 = [out[5] for out in output]

# Plot YUK discharge against cost
scatter(cost, yuk_dis; xlabel="Daily Cost (dollars/day)", ylabel="Daily YUK discharge (kg/da
title="Dirichlet Distribution: Cost vs YUK Discharge", label="Distribution")
hline!([20], linestyle=:dash, color=:red, label="Effluent Standard")
```


Dirichlet Distribution: Cost vs YUK Discharge
Distribution
Effluent Standard
Daily YUK discharge (kg/day)
Daily Cost (dollars/day)
0
20
40
60
80
0
1000
2000
3000
4000